

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Zeichen und Strings

- Stringvariablen und Stringkonstanten

- Arbeiten mit Strings

- Ein-/Ausgabe von Zeichen

- Ein-/Ausgabe von Strings

- Klassifizierung von Zeichen

Null-Zeichen

Stringvariablen werden durch Felder von **char**-Werten realisiert.

Ein Feld von Zeichen wird zu einem String, indem man hinter dem letzten „echten“ Zeichen ein **Null-Zeichen** speichert.

Standard. Null-Zeichen

- ein **char** (1 Byte = **CHAR_BIT** Bits), wobei alle Bits auf 0 gesetzt sind,
- Ordnungszahl 0,
- im Quelltext repräsentiert durch `\0`.

Bei Stringkonstanten sorgt der Compiler automatisch für das Anhängen des Null-Zeichens.

Strings mit einem Zeichen

Jetzt wird klar, warum `'w'` und `"w"` nicht dasselbe bedeuten.

- `'w'` hat als Wert die Ordnungszahl des Zeichen `w`.
- `"w"` hat als Wert die Zeichenfolge aus dem Zeichen `w` und einem nachfolgenden Null-Zeichen.

Deklaration von Stringvariablen

Deklariert man Stringvariablen, so muss man bei der Längenangabe das Null-Zeichen stets mitzählen.

Beispiel

```
char text1[4] = "Text";           // nicht korrekt
```

```
char text2[5] = "Text";
```

```
char text3[6] = "Text";
```

```
char text4[] = "Text";
```

Die erste Deklaration ist **nicht korrekt**, weil sie das abschließende Null-Zeichen **nicht** berücksichtigt.

Bei der zweiten Deklaration sind die Länge von Variable und Konstante gerade identisch.

Die dritte Deklaration ist zulässig; die letzte der sechs verfügbaren Zeichenpositionen bleibt frei.

Es ist ebenfalls möglich den Compiler die Anzahl der benötigten Zeichen ermitteln zu lassen und, wie in der vierten Deklaration, auf die Angabe einer Länge zu verzichten.

Lange Stringkonstanten

Längere Stringkonstanten kann man, in Teile zerlegt, in mehrere Zeilen schreiben.

Man beendet die Teile am Zeilenende mit Anführungszeichen und leitet den nächsten Teil wieder mit Anführungszeichen ein; zwischen den Teilen dürfen neben dem Zeilenende-Zeichen beliebig viele andere white spaces stehen.

Der Compiler fügt die Teile unmittelbar hintereinander.

Beispiel

```
printf("Dieses ist ein nicht besonders langer String\n");  
printf("Dieses ist ein nicht besonders "  
      "langer String\n");
```

Das Resultat ist das gleiche.

Zeichen und Strings

Stringvariablen und Stringkonstanten

Arbeiten mit Strings

Ein-/Ausgabe von Zeichen

Ein-/Ausgabe von Strings

Klassifizierung von Zeichen

Arbeiten mit Strings

Das Null-Zeichen erlaubt Folgendes.

- Das Ende eines Strings lässt sich erkennen, ohne dass man seine Länge vorab kennt.

Die Funktionen `scanf` und `printf` sind zum Beispiel darauf angewiesen, weil sie die Länge ihres Formatierungsstrings ja nicht mitgeteilt bekommen.

- Ein String kann seine Länge während der Ausführung des Programms dynamisch ändern, da er ohne weiteres **kürzer** sein darf als die Stringvariable, in der er gespeichert ist.

Ein String darf, **einschließlich des abschließenden Null-Zeichens**, die Länge des Feldes nie übersteigen, denn Stringvariablen sind Felder, somit gelten auch dieselben Regeln wie für Felder.

Der Programmierer selbst ist dafür verantwortlich, dass keine Indexüberschreitungen vorkommen.

Für das Anhängen des abschließenden Null-Zeichens an einen String ist, abgesehen von den Stringkonstanten und einigen Standardfunktionen, der Programmierer selbst verantwortlich.

Beispiel

Folgende Anweisungsfolge, kopiert einen String. Dabei seien **source** und **target** Felder, wobei **target** groß genug ist um den zu kopierenden String aufzunehmen.

```
i = 0;
while (source[i] != '\0') {
    target[i] = source[i];
    i++;
}
target[i] = '\0';
```

Kürzer kann man schreiben.

```
i = 0;
while (target[i] = source[i]) {
    i++;
}
```

Die Vorteile der zweiten Formulierung gegenüber der ersten.

- Die Schleife terminiert erst, wenn das Null-Zeichen bereits übertragen ist, so dass es nicht hinter der Schleife besonders behandelt werden muss.
- In jedem Schleifendurchlauf wird nur zweimal auf eine Vektorkomponente zugegriffen und nicht dreimal.

Zeichen und Strings

Stringvariablen und Stringkonstanten

Arbeiten mit Strings

Ein-/Ausgabe von Zeichen

Ein-/Ausgabe von Strings

Klassifizierung von Zeichen

Ein-/Ausgabe von Zeichen

Ein Teil der Standardbibliothek von C ist der Verarbeitung von Zeichen und Strings gewidmet.

Es gibt z.B. spezielle Funktionen zur Ein- und Ausgabe von Zeichen. Um diese Funktionen zu nutzen wird `stdio.h` in den Quellcode eingebunden.

Funktionen für Zeichen

Die Funktion `getchar()` liefert als Funktionswert das nächste Zeichen von der Standardeingabe – oder ggf. den Wert `EOF`.

Der Funktionswert von `getchar()` ist `int`. Damit lassen sich alle Zeichen und zusätzlich `EOF` darstellen lassen. Deshalb muss der Funktionswert bei der Zuweisung an eine Zeichenvariable umgewandelt werden. Nach der Umwandlung ist kein Vergleich mit `EOF` mehr möglich, da `EOF` nicht im Wertebereich von `char` liegt.

Die Funktion `putchar(c)` schreibt das Zeichen `c` auf die Standardausgabe.

Dabei ist zu beachten, dass `c` den Typ `int` besitzt oder in den Typ `int` umgewandelt wird, wenn es den Typ `char` oder `short` besitzt.

Die Funktion selbst wandelt diesen Wert in den Typ `unsigned char` zurück, um das zu schreibende Zeichen zu bestimmen.

Der Funktionswert ist das geschriebene Zeichen im Typ `int`, bzw. `EOF` wenn ein Fehler aufgetreten ist.

Beispiel

Folgende Anweisungsfolge, kopiert die Standardeingabe auf die Standardausgabe.

```
#include <stdio.h>

int c;
while ((c = getchar()) != EOF) {
    putchar(c);
}
```

Diese Anweisungsfolge ergibt **kein** Durcheinander auf der Standardausgabe.

- `getchar` arbeitet genau wie `scanf` gepuffert, d.h. es wird zunächst das ENTER des Benutzers abgewartet und dann die eingegebene Zeile Zeichen für Zeichen abgearbeitet.
- `getchar` liefert das Zeilenende-Zeichen (wie alle anderen Zeichen) an das Programm, das es mit `putchar` wieder schreibt.

Mit der Funktion `getchar` hat man auch eine Möglichkeit, bei der Eingabe mehrerer Zahlen den Benutzer dazu zu zwingen, jede Zahl einzeln mit ENTER abzuschließen.

Nachdem die erste Zahl der Zeile interpretiert wurde, wird der Rest der Zeile (bis zum Zeilenende-Zeichen) in einer Schleife übersprungen.

Kopieren von 10 Zahlen (Eine Zahl pro Zeile) (1/2)

```
1  /*****
2  *   read N numbers (one number per line)
3  *****/
4  #include <stdio.h>
5  #define N 10                                // iterations
6
7  int main()
8  {
9      int i = N, value;
10
11     printf("Enter one value per line please.\n");
12
13     while (i--) {                            // loop N-times
14         scanf("%d", &value);                 // read value
15         while (getchar () != '\n') {         // ignore line tail
16             }
17
18         printf("%d\n", value);                // write value
19     }
20     return 0;
21 }
```

Kopieren von 10 Zahlen (Eine Zahl pro Zeile) (2/2)

```
14     scanf("%d", &value);           // read value
15     while (getchar () != '\n') {    // ignore line tail
16     }
```

Es wird ausgenutzt, dass `scanf` die Zeile nur soweit abarbeitet, bis ein zum ersten Zeichen, das **nicht** mehr interpretiert wurde, also mindestens das Zeilenende-Zeichen *überläßt*.

Die *Wegwerf*-Schleife wird mindestens einmal durchlaufen, holt also zumindest ein Zeichen. Wenn die Zeile kein Zeichen mehr enthalten würde, würde über das Betriebssystem eine neue Eingabezeile anfordern, das wäre hier ein vollständig unerwünschter Effekt.

Zeichen und Strings

Stringvariablen und Stringkonstanten

Arbeiten mit Strings

Ein-/Ausgabe von Zeichen

Ein-/Ausgabe von Strings

Klassifizierung von Zeichen

Ein-/Ausgabe von Strings

Für die Ein-/Ausgabe von Strings stehen spezielle Funktionen zur Verfügung.

Sowohl mit den Funktionen `scanf` und `printf` als auch mit den Funktionen `getchar` und `putchar` können Strings gelesen und geschrieben werden.

Zur Erinnerung. Der Formatbeschreiber für Ein- und Ausgabe ist `%s`.

Beispiel, Ausgabe

```
#define N 32

char str[N];
int i;

/* ... init string ... */

printf ("%s", str);                // variation 1

i = 0;                             // variation 2
while (str[i])
    putchar(str[i++]);
```

Die Schleife kann nur korrekt funktioniert, wenn der String durch ein Null-Zeichen abgeschlossen ist.

Eingabe von Strings

Beispiel, Eingabe

```
#define N 32

char str[N];
int i;

scanf ("%s", str);                // variation 1
scanf ("%s", &str[0]);           // variation 1A

i = 0;                            // variation 2
while ((str[i] = getchar()) != '\n')
    i++;
str[i] = '\0';
```

Die Eingaben sind nicht ganz äquivalent.

- **scanf** überliest zunächst „white spaces“ und liest danach Zeichen, bis ein erneutes „white space“ gefunden wird.
- In der Schleife wird auf jeden Fall von der aktuellen Position in der Zeile bis zum Zeilenende gelesen. Die Anweisung hinter der Schleife ersetzt das Zeilenende-Zeichen durch das Null-Zeichen.

In beiden Fällen ist nicht sichergestellt, dass der gelesene String in die Variable **str** hineinpasst.

Sichere Eingabe

Um den Aufruf von `scanf` sicherer zu machen, benötigt man die erweiterten Möglichkeiten von Formatbeschreibern, das muss noch etwas warten.

Die Schleife kann man jedoch leicht geeignet modifizieren.

```
#define N 32

char str[N];
int i;

i = 0;
while (i < N && (str[i] = getchar()) != '\n')
    i++;

if (i == N) {
    printf("ERROR: buffer overflow\n");
    return 1;
}
else {
    str[i] = '\0';
}
```

Zeichen und Strings

Stringvariablen und Stringkonstanten

Arbeiten mit Strings

Ein-/Ausgabe von Zeichen

Ein-/Ausgabe von Strings

Klassifizierung von Zeichen

Klassifizierung von Zeichen

Standardfunktionen zur Verarbeitung von Zeichen und Strings stellen neben `stdio.h` auch `ctype.h` und `string.h` zur Verfügung.

- Headerdatei `ctype.h` definiert Funktionen zur Klassifizierung von Zeichen und zur Umwandlung zwischen Groß- und Kleinbuchstaben.
- Headerdatei `string.h` definiert Funktionen mit denen man
 - ▶ Strings kopieren, konkatenieren oder vergleichen,
 - ▶ in Strings nach Zeichen oder Strings suchen oder
 - ▶ verschiedene andere Dinge erledigen kann.

ctype.h

Der Umfang der Headerdatei `ctype.h` ist nicht groß und die Funktionen sind sehr grundlegend und einfach.

Die Funktionen `tolower` und `toupper` wandeln Groß- in Kleinbuchstaben bzw. Klein- in Großbuchstaben um; andere Zeichen bleiben unverändert.

Das Argument ist jeweils das ggf. umzuwandelnde Zeichen in `int`-Darstellung, der Funktionswert das resultierende Zeichen in `int`-Darstellung.

Die übrigen Funktionen haben als Argument ebenfalls ein Zeichen in `int`-Darstellung; ihr Funktionswert ist ungleich oder gleich Null und kann entsprechend als *wahr* oder *falsch* interpretiert werden, je nachdem, ob das Zeichen zu einer bestimmten Gruppe von Zeichen gehört oder nicht.

Funktionen zur Klassifizierung von Zeichen (`ctype.h`)

Funktion	Beschreibung
<code>islower</code>	Kleinbuchstabe
<code>isupper</code>	Großbuchstabe
<code>isalpha</code>	Buchstabe, klein oder groß
<code>isdigit</code>	Dezimalziffer
<code>isxdigit</code>	Hexadezimalziffer
<code>isalnum</code>	Buchstabe oder Ziffer
<code>iscntrl</code>	Steuerzeichen (nicht druckbar)
<code>isgraph</code>	druckbares Zeichen (ohne Leerzeichen)
<code>isprint</code>	druckbares Zeichen (einschließlich Leerzeichen)
<code>ispunct</code>	druckbares Sonderzeichen (ohne Leerzeichen)
<code>isspace</code>	„white space“

Allen Funktionen (einschließlich `tolower` und `toupper`) darf man als Argument auch `EOF` übergeben und erhält dann `EOF` als Funktionswert zurück.

Beispiel (1/2)

Einlesen einer Hexadezimalzahl als Zeichenfolge und Umwandeln in die entsprechende interne Darstellung.

Wir gehen davon aus, dass die eingegebene Zahl nicht größer als `UINT_MAX` ist.

Die Interpretation soll stoppen, sobald das erste nicht interpretierbare Zeichen gefunden wird.

Diese Aufgabe könnte man zwar auch mit `scanf` lösen, es ist aber nützlich sich Gedanken darüber zu machen was bei der Ausführung einer Standardfunktion tatsächlich passiert.

Beispiel (2/2)

```
#include <stdio.h>
#include <ctype.h>
...
    int c;
    unsigned int x, h;

    char hex_digit[] = "0123456789ABCDEF";

    x = 0;
    while (isxdigit(c = getchar())) {
        x *= 16;
        h = 0;
        while (hex_digit[h] != toupper(c))
            h++;
        x += h;
    }
    ...
```

Die Anweisungsfolge leistet das Verlangte und ist ohne weiteres portierbar auf Implementationen, denen z.B. nicht der ASCII-Zeichensatz zugrunde liegt.

Es ist keinerlei Sicherung gegen eine Bereichsüberschreitung eingebaut.